

Microsoft Work IQ Hackathon

Build on the Work Graph. Answer what no single app can.

Welcome to the **Work IQ Hackathon**. Over the next two days, six teams will each take on one fictional customer whose people waste hours every day stitching answers together by hand — hopping between Outlook, Teams, SharePoint, OneDrive, the calendar, and line-of-business systems — because **no single application can see the whole picture of work**.

Your job is to use **Microsoft Work IQ** — the reasoning layer over the Microsoft 365 work graph — to turn that fragmented, multi-app reality into a single grounded, cited, permission-trimmed answer. Where the Foundry IQ hackathon was about designing a *knowledge layer* over static documents, this hackathon is about grounding agents in **live work context**: the emails, meetings, chats, files, people, and collaboration signals that already exist in a tenant, plus the Dataverse records and Copilot connector data your customer brings.

You will design *which* work signals to ground in, choose *how* to reach Work IQ (A2A, MCP, or REST), light up all four Work IQ capability domains (**Context, Chat, Tools, Workspaces**), build a working agent, and demonstrate one production-grade governance control. Then you'll pitch it.

Read this first. Every scenario in this pack is achievable with Work IQ today. Work IQ grounds in **work context the user already has access to** — it does not crawl public document corpuses, it does not replace your line-of-business database, and it always honours the signed-in user's permissions and sensitivity labels. If a question can only be answered by static reference documents with no work-context signal, it is *not* a Work IQ question — and you won't find one in this pack.

How the Hackathon Works

Each team owns one challenge end to end. The seven steps below are the spine of every challenge and the backbone of your pitch.

1. Design your work-context grounding strategy. Decide *which* Microsoft 365 work signals your scenario depends on — email threads, meeting transcripts and recaps, Teams chats and channel posts, shared and recently-touched files, the people graph (who works with whom, reporting lines, expertise), calendar and availability — plus any **Dataverse tables** or **Copilot connector** sources your customer brings (e.g. a CRM, an ITSM queue, an HRIS). Map each of the customer's hard questions to the signals that answer it. This map is your architecture, and it is the single most important deliverable.

2. Configure your Work IQ access path. Choose how your agent reaches Work IQ and justify it:

- **A2A (Agent-to-Agent)** (*preview*) — your agent **discovers** the Work IQ agent from its **A2A agent card** (`workiq agents list` → `workiq agents get-card`), then sends an A2A `message/send` request to the `url` advertised on that card (`preferredTransport: JSONRPC` , `protocolVersion 0.3.0` , skill `ask_work_iq`), authenticating with the `WorkIQAgent.Ask` scope; best for "ask a natural-language work

question, get a grounded, cited answer." Requires the **preview** Work IQ CLI (`@microsoft/workiq@preview`); the `agents` commands are not in the stable build.

- **MCP (Model Context Protocol)** — connect to the Work IQ MCP server and call its generic tools (entity, copilot, schema categories) for structured fetch/act workflows.
- **REST (preview)** — direct API calls for custom integrations where you control the orchestration.

Then, if you're building a **long-running or multi-step agent**, give it a **Workspace** — a tenant-bounded scratch space where it persists files, memory, and intermediate reasoning across turns without that state ever leaving the Microsoft 365 trust boundary. (Note: a Workspace is *not* how you restrict what work context the agent can read — that is always trimmed to the signed-in user's Microsoft 365 permissions and sensitivity labels, enforced by the platform, not the workspace.)

3. Demonstrate work-context retrieval (Context). Show Work IQ answering a **single-signal** question (grounded in one signal type — e.g. "what did my manager email me about the launch?") and a **multi-signal** question (synthesising two signal types — e.g. email + meeting). Every answer must carry **citations back to the underlying work items**.

4. Demonstrate agentic synthesis. Show Work IQ decomposing a genuine **compound question** that spans **three or more** signals and returning one synthesised, cited answer — the question a human would otherwise answer by opening four apps and reading for twenty minutes. You can do this two ways, and the choice is itself an architecture decision to justify: call **Chat** (`ask`) and let Work IQ return a finished, cited answer, or pull grounded sources via **Context** (`fetch`) and have *your own* model do the synthesis.

5. Build an Agent with Work IQ (MCP / A2A) + a Tools action. Stand up a conversational agent that uses Work IQ as a tool. It must not only *read* context — it must take **one governed action** via Work IQ **Tools** (e.g. `create_entity`, `update_entity`, or `do_action` against Dataverse / a connector — draft a follow-up, create a task, update a record). Reads inform; the action proves it can close the loop.

6. Demonstrate governance. Show one production-realistic control. Options: **permission trimming** (a second persona asks the same question and is correctly denied data they can't see — Work IQ never widens the signed-in user's access), **sensitivity-label honouring** (a labelled document is excluded or redacted), or **admin consent / least-privilege** (only `WorkIQAgent.Ask` is granted, and the agent provably cannot exceed the signed-in user's permissions). The control must be *demonstrated live*, not described.

7. Deliver a Working Demo + Pitch Deck. Two artefacts: **(1) a Working Demo** — a live or recorded walkthrough answering at least one compound question and performing one Tools action against your work-context architecture; and **(2) a Pitch Deck** (5–8 slides) telling the before/after story: the customer's pain, the work signals you grounded in, the questions now answerable with citations, the governance control, and the measurable business outcome.

Two things every team must surface in the pitch: the **tenant-authorization requirements** your lab depends on (see each challenge's *Tenant authorization* callout), and — if you ran without full tenant access — which parts of your demo used the **Local Work IQ Simulator** (see the *Alternate Path* section at the end of this pack).

Scoring: Work-context architecture design (30%) · Retrieval & synthesis quality (25%) · Work IQ capability breadth (25%) · Business impact narrative (20%). Full rubric at the end.

Challenge 1 — Northbridge Health Network

"Running a regional health network is a coordination problem. The answers are spread across a hundred meetings and inboxes."

The customer

Northbridge Health Network operates three hospital campuses and fourteen outpatient clinics across a regional health authority. This challenge is about the **non-clinical knowledge work** of running the network — operations, quality programs, credentialing, vendor management, regulatory committee work — *not* patient care. The people who run Northbridge live in a blizzard of cross-functional committees: the quality steering committee, the credentialing committee, the EHR-rollout program, and the regulatory readiness group. Every one of them generates meetings, email threads, shared documents, and Teams discussion — and the institutional memory of *what was decided and why* lives nowhere a person can search.

The situation

A quality program manager preparing for a Joint Commission readiness review needs to know what the quality steering committee decided about the medication-reconciliation policy three meetings ago, who was assigned the corrective action, and whether the credentialing committee ever closed the related provider-onboarding gap. Those answers live in meeting recaps, an email thread, and a shared tracking file — across two committees whose notes the program manager *can* access but has no time to reconcile by hand. A new operations director cannot get a straight answer to "what's the status of the EHR rollout at the Westgate clinic" because the status is distributed across the program channel, the last steering meeting, and the vendor's emails.

What the customer needs answered — Microsoft Work IQ

Work IQ capability	Key question the customer cannot answer today
Work Context — single signal	"What did the quality steering committee decide about the medication-reconciliation policy in its last meeting, and who owns the follow-up?" — grounded in meeting recap + action items.
Work Context — single signal	"What has the EHR vendor emailed us about the Westgate go-live date, and what did we ask them to confirm?" — grounded in the email signal scoped to the vendor.
Work Context — multi-signal	"What is the current status of the EHR rollout at Westgate — combine the program channel updates with the last steering-committee meeting." — Teams channel + meetings.
Work Context — multi-signal	"Which corrective actions from the last two quality meetings are still open , and what do the email threads say about why they're stalled?" — meetings + email.

Work IQ capability	Key question the customer cannot answer today
Agentic synthesis	"Prep me for the Joint Commission readiness review : what did the quality committee decide on med-reconciliation, who owns the corrective action, did credentialing close the related onboarding gap, and what's outstanding?" — meetings (two committees) + email + the shared tracking file, synthesised.
Agentic synthesis	"Give me a cross-committee view of everything touching the Westgate clinic in the last month — decisions, open actions, vendor commitments, and the people accountable." — synthesis across all committee signals, scoped to one site.
Agent with MCP / A2A	An operations director asks: "For every open corrective action from the quality committee, update its status in our program tracker and flag the ones past due." — agent reads via Context, acts via Tools (<code>update_entity</code> against the Dataverse/connector CAPA tracker).
Agent with MCP / A2A	"Draft the readiness-review summary memo for leadership pulling decisions, open actions, and owners across both committees" — agent synthesises and uses Tools to draft the document.

Work context hints

Think about: **meetings** (committee recaps, decisions, action items, attendees across multiple governance bodies), **email** (vendor correspondence, leadership threads, escalations), **Teams channels** (program rollout channels per clinic), **files** (the shared corrective-action tracker, policy drafts), the **people graph** (committee membership, who owns which corrective action), and a **program/CAPA tracker** in Dataverse or a connector surfaced via Tools. The hard questions stitch two *different committees*' meetings to an email thread and a tracker record. Note: this is administrative coordination — no PHI or clinical decision-making is in scope.

The north-star outcome

A quality program manager walks into the readiness review fully prepared in five minutes — every decision, owner, and open action across two committees in one cited brief — and the agent has already updated the corrective-action tracker so leadership sees real status, not a stale spreadsheet.

 **Tenant authorization required:** Copilot licenses for committee members, **admin consent for WorkIQAgent.Ask**, Work IQ enabled in MAC, and a Dataverse/connector CAPA tracker with write access for the Tools action. Sensitivity labels on policy documents must be honoured by the demo. Run without tenant access on the **Local Work IQ Simulator** (Alternate Path §C1).

Challenge 2 — Contoso Precision Parts

"The program status lives in 200 emails, 40 meetings, and a dozen channels. Nobody can see it whole."

The customer

Contoso Precision Parts manufactures automotive and aerospace components across three production floors with 400+ CNC machines. A single new-part program — say the **Part 45621-B** introduction on the **DMG Mori DMU-14** line — runs for months across engineering, supply chain, quality, and program management, with 60+ people contributing. The authoritative record of *what is actually happening* is not in any system of record. It is scattered across the program manager's inbox, the weekly design- and production-readiness meeting recaps, three Teams channels, the shared OneDrive of working documents, and a Dataverse-based **program milestone tracker**. When a program manager is out — or hands the program to a successor — that context does not transfer.

The situation

A new program manager inheriting the 45621-B program needs to reconstruct, from scratch, what was decided in the last design review, which supplier risk is currently open on the Inconel 718 bar stock, who owns the PPAP qualification plan, and what the customer's last escalation was about. Today that means a week of reading old email threads, scrubbing meeting recordings, and asking around. A manufacturing engineering lead asks "what's blocking the 45621-B qualification?" and the answer is split between a quality engineer's email, a decision made verbally in last Thursday's review, and a risk logged in the milestone tracker. Nobody can answer it without three people in a room.

What the customer needs answered — Microsoft Work IQ

Work IQ capability	Key question the customer cannot answer today
Work Context — single signal	"What did the last 45621-B design review meeting conclude, and what action items came out of it with owners?" — grounded in meeting recap + transcript signals.
Work Context — single signal	"What did our customer's program office email us about in the last escalation thread, and what did we commit to?" — grounded in the email signal, scoped to the customer's domain.
Work Context — multi-signal	"What is currently blocking the 45621-B qualification — combine the open action items from the design-review meetings with the risk emails the quality team sent to the program distribution list." — meetings + email synthesised.
Work Context — multi-signal	"What's been said about the Inconel 718 supplier issue across the team's chats and email in the last month, and who has been driving it?" — chat + email.
Agentic synthesis	"I'm taking over the 45621-B program — give me the current state: last design-review decisions, open supplier risks, who owns the PPAP qualification plan, and the customer's most recent escalation." — meetings + email + chat + the Dataverse milestone tracker, synthesised into one cited handover brief.
Agentic synthesis	"Across the program, what changed in the last two weeks that I should know about — decisions, new risks, slipped milestones, and people who flagged concerns?" — a recency-scoped synthesis across all signals.

Work IQ capability	Key question the customer cannot answer today
Agent with MCP / A2A	A program manager asks the agent: "Summarise the open qualification blockers and create a tracked risk item in the milestone tracker for each one, assigned to the named owner." — agent reads via Work IQ Context, then uses Tools (<code>create_entity</code> against the Dataverse tracker) to act.
Agent with MCP / A2A	"Draft the weekly program status email to the customer covering decisions, risks, and milestone movement since the last update" — agent synthesises work context and uses Tools to draft (not send) the outgoing mail for review.

Work context hints

Think about: **email** (program threads, customer escalations, supplier correspondence — sender, recipients, thread, date, attachments), **meetings** (design- and production-readiness recaps, transcripts, action items, attendees), **Teams chats and channel posts** (the Inconel-718 supplier channel, ad-hoc decisions), the **people graph** (who owns what, expertise, reporting lines), **files** (the working PPAP qualification plan, supplier risk register), and a **Dataverse milestone tracker** surfaced through Work IQ Tools (milestone, status, owner, risk fields). The hardest questions span a *verbal decision in a meeting*, a *risk in email*, and a *milestone in Dataverse* simultaneously. How you let Work IQ correlate program, part number, supplier, and owner across signals determines whether the handover brief is complete or full of holes.

The north-star outcome

A new program manager opens the agent on day one, asks "bring me up to speed on 45621-B," and receives — in under a minute, with citations back to the exact meetings, emails, and records — a complete handover brief, then asks the agent to open risk items for every open blocker. A week of catch-up becomes a single conversation.

⚠ Tenant authorization required: Grounding in real meetings/email/chat requires each participant's **M365 Copilot license**, **admin consent for** `WorkIQAgent.Ask`, **Work IQ enabled in the Microsoft Admin Center**, and a **Dataverse environment** with the milestone table. For the Tools write-back (`create_entity`), Work IQ is **delegated-only** — the **signed-in user (or test persona)** must have write access to that Dataverse table; there is no app-only/service-principal write path. Teams with no tenant access run this on the **Local Work IQ Simulator** (see Alternate Path §C2).

Challenge 3 — Meridian Group

"Every consultant carries their client in their head. When they're out, the client falls off a cliff."

The customer

Meridian Group is a 9,000-person professional services firm operating across 14 countries, running 30–40 concurrent client engagements at any time. Each engagement is a dense web of work context: the

statement of work and deliverable files in SharePoint, the client correspondence in Outlook, the delivery-team standups and decisions in Teams, the project plan and status in a Dataverse-based **engagement tracker**, and the billing/utilisation data in a connector to their PSA (professional services automation) system. An engagement manager holds all of this in their head. When they take leave — or roll off — the next person starts blind.

The situation

A delivery principal covering for a colleague on leave needs an instant **360° view** of the Brightwater client: what's the current sprint delivering, what did the client last escalate, which deliverables are due this week, what's the open scope-change discussion, and is the engagement on budget. Those answers are spread across email, three Teams channels, the SOW files, the engagement tracker, and the PSA. A partner walking into a client QBR cannot quickly answer "what have we actually delivered this quarter and what's at risk" without three analysts pulling it together for a day.

What the customer needs answered — Microsoft Work IQ

Work IQ capability	Key question the customer cannot answer today
Work Context — single signal	"What did the Brightwater client last escalate over email, and what did we commit to in response?" — grounded in the email signal scoped to the client.
Work Context — single signal	"What did the last Brightwater standup cover, and what did the team commit to for this sprint?" — grounded in the standup meeting / Teams signal.
Work Context — multi-signal	"Is there an open scope-change discussion on Brightwater, and where does it stand across the client emails and the internal delivery channel?" — email + Teams chat.
Work Context — multi-signal	"What deliverables are due this week for Brightwater, and what do the team's chats say about whether we'll hit them?" — the engagement tracker + Teams chat.
Agentic synthesis	"Give me a 360 on Brightwater : current sprint, last client escalation, deliverables due this week, the open scope change, and budget status." — email + Teams + SOW files + engagement tracker + PSA connector, synthesised into one cited account brief.
Agentic synthesis	"Across all my engagements , which clients have an open escalation, a deliverable at risk this week, or a budget overrun — and who on my team owns each?" — portfolio-level synthesis across every engagement's signals.
Agent with MCP / A2A	A delivery principal asks: "For the Brightwater scope change, create a change-request record in the engagement tracker with the discussed scope and estimated effort, owned by the lead consultant." — agent reads via Context, acts via Tools (<code>create_entity</code>).
Agent with MCP / A2A	"Draft the Brightwater QBR status update covering delivered work, risks, and budget" — agent synthesises and uses Tools to draft the deck outline / document.

Work context hints

Think about: **email** (client correspondence, escalations, scope discussions), **meetings and Teams chats** (standups, delivery decisions, internal channels per client), **files** (SOWs, deliverables, status decks in

SharePoint), the **people graph** (engagement manager, lead consultant, who knows the client), a Dataverse **engagement tracker** (project, sprint, deliverable, due date, status) and a **PSA connector** (budget, utilisation, billing) — both via Tools. The hardest questions need the *client's words* (email), the *team's reality* (chats + tracker), and the *commercial truth* (PSA) together. Index client, engagement, deliverable, and owner consistently so cross-engagement portfolio questions resolve.

The north-star outcome

A delivery principal covers a colleague's client with zero ramp-up: one question yields a full account 360 with citations, and the agent opens the scope-change record on the spot — so the client never feels the handoff.

 **Tenant authorization required:** Copilot licenses for the delivery team, **admin consent for WorkIQAgent.Ask**, Work IQ enabled in MAC, a Dataverse engagement tracker, and a **Copilot connector** to the PSA system (separate admin setup + connector credentials). Work IQ is delegated-only — the **signed-in user (or test persona)** must have write access to the tracker for the Tools action. No tenant access → **Local Work IQ Simulator** (Alternate Path §C3).

Challenge 4 — Arundel Holdings

"Forty buildings, two asset systems, and a vendor for everything. The coordination lives in inboxes."

The customer

Arundel Holdings manages commercial real estate and facilities operations across 40 buildings in 6 cities. This challenge is about the **operational coordination** work of running the portfolio — fault response, planned-maintenance campaigns, vendor and contractor management, tenant fit-outs and building turnovers — *not* the raw asset telemetry. When a major building event happens — a chiller failure at a flagship site, a planned shutdown overrunning — the response spins up a cross-functional storm of work context: the facilities coordination meetings, the operations and engineering email, the building-ops Teams channel, the shared works log file, and a Dataverse-based **work-order tracker** (with ServiceNow and Maximo data surfaced through a connector). The decisions and commitments that matter get made fast and recorded nowhere searchable.

The situation

A facilities duty manager taking over coverage mid-event needs to know what the last coordination call decided about the **Calder Tower** chiller failure, what the engineering team emailed about the suspected cause, which contractor was dispatched and what their ETA is, and what tenant-notification commitment applies. Those live across coordination-meeting recaps, email, the building-ops channel, the works log, and the work-order tracker. A portfolio operations manager doing the monthly vendor review has to reconstruct, by hand from five sources, which vendor is behind on which buildings.

What the customer needs answered — Microsoft Work IQ

Work IQ capability	Key question the customer cannot answer today
Work Context — single signal	"What did the last facilities coordination call decide about the Calder Tower chiller failure, and what actions were assigned?" — grounded in meeting recap + transcript.
Work Context — single signal	"What has the mechanical contractor emailed us about the Calder Tower repair scope and their ETA?" — grounded in the email signal scoped to the vendor.
Work Context — multi-signal	"Which work orders are open at Calder Tower and what do the building-ops channel posts say about progress on each?" — the work-order tracker + Teams chat.
Work Context — multi-signal	"What's the current status of the Calder Tower restoration across the last coordination call and the shared works log file?" — meetings + files.
Agentic synthesis	"I'm taking coverage on the Calder Tower chiller event — give me the current picture: last coordination-call decisions, suspected cause, contractor dispatched and ETA, open work orders and owners, and any tenant-notification commitment." — meetings + email + chat + works log + work-order tracker, synthesised with citations.
Agentic synthesis	"Across all my buildings , which vendors have an overdue work order, an unanswered escalation email, or a missed planned-maintenance window this month — and who owns each?" — portfolio synthesis across sites and vendors.
Agent with MCP / A2A	A duty manager asks: "For each open action from the last coordination call, update the work order in the tracker with owner, asset, and due date." — agent reads via Context, acts via Tools (<code>update_entity</code>).
Agent with MCP / A2A	"Draft the tenant-notification note and the leadership status update for the Calder Tower event" — agent synthesises and uses Tools to draft both documents.

Work context hints

Think about: **meetings** (facilities coordination calls — recaps, transcripts, decisions, attendees), **email** (engineering cause analysis, vendor correspondence, tenant notifications), **Teams building-ops channel** (contractor dispatch, site updates), **files** (the shared works log, restoration plan), the **people graph** (duty manager, on-call engineer, dispatched contractor, vendor account owner), and a Dataverse **work-order tracker** (work order, asset, building, status, owner, due date) — with ServiceNow and Maximo data surfaced via a **connector** through Tools. The hardest questions stitch a *coordination call decision*, a *channel dispatch*, and a *work-order record* across multiple buildings into one operational picture. Treat building, asset, vendor, and owner as the spine of your correlation.

The north-star outcome

A facilities duty manager takes coverage in two minutes with a complete, cited operational picture across the affected building, and the agent has the work orders updated and the tenant-notification draft ready — turning a chaotic verbal handoff into a controlled, documented one, and the monthly vendor review into a single question.

 **Tenant authorization required:** Copilot licenses for the facilities team, **admin consent for WorkIQAgent.Ask**, Work IQ enabled in MAC, a Dataverse work-order environment, and a **Copilot connector** to the ServiceNow/Maximo systems (separate admin setup + connector credentials). Work IQ is delegated-only — a **signed-in user (or test persona)** must have write access to the work-order table for the Tools action. No tenant access → **Local Work IQ Simulator** (Alternate Path §C4).

Challenge 5 — Westbrook University

"Running a university is committee work. The decisions that matter are buried in a year of minutes."

The customer

Westbrook University is a comprehensive regional institution with 18,000 students, 900 faculty, and 120 academic programs. This challenge is about the **administrative knowledge work** of running the institution — curriculum and assessment committees, accreditation self-studies (AACSB, ABET), program reviews, faculty search committees, and the provost's office — *not* student records or grading. Every governance body generates meetings, email threads, shared self-study and review documents, and Teams discussion, and the institutional memory of *what was decided and why* is scattered across a year of minutes that no one can search.

The situation

An associate dean preparing for an **AACSB reaffirmation** site visit for the College of Business needs to know what the curriculum committee decided about the revised B.B.A. assessment plan two meetings ago, who was assigned to close the "assurance of learning" gap, and whether the program-review committee ever signed off on the related course-sequence change. Those answers live in committee meeting recaps, an email thread, and a shared self-study document — across two committees whose notes the dean *can* access but has no time to reconcile by hand. A new department chair cannot get a straight answer to "where does the B.B.A. program review actually stand" because the status is split across the program channel, the last committee meeting, and the provost office's emails.

What the customer needs answered — Microsoft Work IQ

Work IQ capability	Key question the customer cannot answer today
Work Context — single signal	"What did the curriculum committee decide about the revised B.B.A. assessment plan in its last meeting, and who owns the follow-up?" — grounded in meeting recap + action items.
Work Context — single signal	"What did the accreditation liaison email us about the AACSB site-visit timeline, and what evidence did they ask us to prepare?" — grounded in the email signal scoped to the review.
Work Context — multi-signal	"What is the current status of the B.B.A. program review — combine the program channel updates with the last program-review committee meeting." — Teams channel + meetings.
Work Context — multi-signal	"Which assurance-of-learning actions from the last two committee meetings are still open , and what do the email threads say about why they're stalled?" — meetings + email.
Agentic synthesis	"Prep me for the AACSB site visit : what did the curriculum committee decide on the assessment plan, who owns the assurance-of-learning gap, did the program-review committee sign off the course-sequence change, and what's outstanding?" — meetings (two committees) + email + the shared self-study file, synthesised.
Agentic synthesis	"Give me a cross-committee view of everything touching the B.B.A. program this academic year — decisions, open actions, accreditation commitments, and the people accountable." — synthesis across all committee signals, scoped to one program.
Agent with MCP / A2A	An associate dean asks: "For every open assurance-of-learning action from the curriculum committee, create a tracked task in the accreditation tracker with owner and due date." — agent reads via Context, acts via Tools (<code>create_entity</code>).
Agent with MCP / A2A	"Draft the site-visit readiness summary for the dean's office pulling decisions, open actions, and owners across both committees" — agent synthesises and uses Tools to draft the document.

Work context hints

Think about: **meetings** (curriculum, assessment, program-review, and faculty-search committee recaps — decisions, action items, attendees across multiple governance bodies), **email** (accreditation-liaison correspondence, provost-office threads, escalations), **Teams channels** (per-program review channels), **files** (the shared self-study document, assessment reports, review templates), the **people graph** (committee membership, who owns which assurance-of-learning action), and an **accreditation/action tracker** in Dataverse or a connector surfaced via Tools. The hard questions stitch two *different committees'* meetings to an email thread and a tracker record. Note: this is administrative governance — no FERPA-protected student records or grading decisions are in scope.

The north-star outcome

An associate dean walks into the accreditation site visit fully prepared in five minutes — every decision, owner, and open action across the curriculum and program-review committees in one cited brief — and the agent has already created the outstanding tasks so the dean's office sees real status, not a stale spreadsheet.

 **Tenant authorization required:** Copilot licenses for committee members, **admin consent for WorkIQAgent.Ask**, Work IQ enabled in MAC, and a Dataverse/connector accreditation tracker with write access for the Tools action. Sensitivity labels on self-study and personnel documents must be honoured by the demo. Run without tenant access on the **Local Work IQ Simulator** (Alternate Path §C5).

Challenge 6 — Enterprise Developer Knowledge Hub

"An incident response is a coordination storm. The decisions that matter are buried in the noise."

The customer

A large software engineering organisation (5,000+ engineers) runs hundreds of services in production. This challenge is about the **operational coordination** work of running engineering — incident response, on-call shift handovers, architecture decision-making, and release coordination — *not* the source code or the live telemetry itself. When a major incident happens — a Sev-1 outage of the **Atlas payments service** — the response spins up a cross-functional storm of work context: the incident bridge calls, the engineering and SRE email, the **#atlas-incident** Teams channel, the shared incident document, and a Dataverse- (or Jira-connector-) based **incident & action tracker**. The decisions and commitments that matter get made fast and recorded nowhere searchable, and the next on-call engineer starts blind.

The situation

An on-call engineer taking over a shift mid-incident needs to know what the last bridge call decided about the **Atlas payments outage**, what the SRE team emailed about the suspected root cause, which mitigation was rolled out and by whom, and what the customer-communications commitment is. Those live across bridge-meeting recaps, email, the incident channel, the shared incident doc, and the action tracker. A reliability engineer writing the post-incident review (PIR) has to reconstruct the full timeline by hand from five sources, days later — and the architecture decision that the bridge made under pressure never makes it into an ADR.

What the customer needs answered — Microsoft Work IQ

Work IQ capability	Key question the customer cannot answer today
Work Context — single signal	"What did the last incident bridge call decide about the Atlas payments outage, and what actions were assigned?" — grounded in meeting recap + transcript.
Work Context — single signal	"What has the SRE team emailed the incident distribution about the suspected root cause of the Atlas outage?" — grounded in the email signal scoped to the incident distribution.
Work Context — multi-signal	"Which mitigation was rolled out for Atlas and by whom, across the #atlas-incident channel and the action tracker?" — Teams chat + the tracker entity.
Work Context — multi-signal	"What's the current status of the Atlas restoration across the last bridge call and the shared incident doc?" — meetings + files.
Agentic synthesis	"I'm taking the on-call shift on the Atlas incident — give me the current picture: last bridge-call decisions, suspected root cause, mitigation rolled out and by whom, open actions and owners, and the customer-communications commitment." — meetings + email + chat + incident doc + action tracker, synthesised with citations.
Agentic synthesis	"Build the post-incident timeline for the Atlas outage — decisions, mitigations, actions, and status changes in chronological order with who did what." — chronological synthesis across all incident signals.
Agent with MCP / A2A	An on-call engineer asks: "For each open action from the last bridge call, create or update the action item in the incident tracker with owner, service, and due time." — agent reads via Context, acts via Tools (<code>create_entity</code> / <code>update_entity</code>).
Agent with MCP / A2A	"Draft the shift-handover brief and the post-incident-review starting point for the Atlas outage" — agent synthesises and uses Tools to draft both documents.

Work context hints

Think about: **meetings** (incident bridge calls — recaps, transcripts, decisions, attendees), **email** (SRE root-cause analysis, customer-communications correspondence), **Teams incident channel** (mitigations, engineer updates), **files** (the shared incident doc, runbook, draft PIR), the **people graph** (incident commander, on-call engineer, service owner), and a Dataverse **incident & action tracker** (action item, service, status, owner, due time) — with Jira/PagerDuty data surfaced via a **connector** through Tools. The hardest questions stitch a *bridge-call decision*, a *channel mitigation*, and an *action-tracker record* into one operational picture — and the **chronological timeline** synthesis is where Work IQ's cross-signal reasoning shines.

The north-star outcome

An on-call engineer takes a mid-incident handover in two minutes with a complete, cited operational picture, and the agent has the action items updated and the post-incident-review starting point ready —

turning a chaotic verbal handoff into a controlled, documented one.

⚠️ **Tenant authorization required:** Copilot licenses for the incident team, **admin consent for WorkIQAgent.Ask**, Work IQ enabled in MAC, a Dataverse incident/action environment (or a **Jira/PagerDuty connector**), and — Work IQ being delegated-only — a **signed-in user (or test persona)** with write access to the action-tracker table for the Tools action. No tenant access → **Local Work IQ Simulator** (Alternate Path §C6).

Hackathon Judging Criteria

Criterion	Weight	What judges are looking for
Work-context architecture design	30%	Did the team identify the <i>right</i> M365 work signals (email, meetings, chats, files, people) plus Dataverse/connector sources for each question? Is the mapping from customer questions to signals explicit and complete? Is the access path (A2A / MCP / REST) chosen deliberately? Does the architecture actually enable the compound questions — or does it leave gaps a human would still have to fill?
Retrieval & synthesis quality	25%	Does Work IQ genuinely decompose compound questions and synthesise across multiple signals, not just surface a single email? Are answers grounded with citations back to the underlying work items (the specific meeting, email, file, record)? Is the synthesis accurate and free of hallucinated commitments? Does it correctly honour recency and scope?
Work IQ capability breadth	25%	Did the team activate the core capability domains — Context (single + multi-signal grounding, the two <i>Work Context</i> tiers), Chat (agentic synthesis via <code>ask</code>), and Tools (at least one governed <code>create/update/do_action</code> , the <i>Agent with MCP / A2A</i> tier) — and, where they built a long-running agent, use Workspaces for tenant-bounded scratch state? Did they choose the right protocol (A2A / MCP / REST) with justification, use Work IQ as a tool correctly via MCPTool / A2A , and enforce the right system-prompt behaviours (cite, don't fabricate, respect the signed-in user's permissions)?
Business impact narrative	20%	Two artefacts: (1) a Working Demo answering ≥ 1 compound question and performing ≥ 1 Tools action live against the work-context architecture; (2) a Pitch Deck (5–8 slides) telling the before/after story — the pain, the signals grounded in, the questions now answerable with citations, the governance control, and the measurable outcome (hours saved, handoffs de-risked). Would the pitch convince a real customer to deploy Work IQ?

A Note on Work IQ Capabilities

Capability	What it is	Best for	What it is not
Context	Grounded retrieval and reasoning over the signed-in user's Microsoft 365 work graph — email, meetings, chats, files, people, calendar, collaboration signals — plus connected Dataverse and Copilot connector data, all permission-trimmed.	Answering "what is the state of <i>my</i> work / <i>this</i> program / <i>this</i> client" by synthesising the signals the user already has access to. The foundational layer — every other capability grounds on it.	A public document search engine, a data warehouse, or a replacement for your line-of-business system. It reasons over work context the user can already see; it never widens their access.
Chat	Natural-language Q&A against work context, and the ability to invoke Copilot agents (<code>ask</code> , <code>agents list</code>) — the conversational front door to Work IQ.	Letting a person (or your agent) ask a compound work question in plain English and get a synthesised, cited answer.	A general-purpose chatbot. Its answers are only as good as the work-context grounding beneath them and the user's permissions.
Tools	A generic, governed tool surface over entities and actions — <code>fetch/create/update/delete entity</code> , <code>do_action</code> , <code>call_function</code> , plus schema discovery (<code>get_schema</code> , <code>search_paths</code>) — typically over Dataverse and Copilot connectors.	Closing the loop: not just <i>reading</i> context but <i>acting</i> on it (create a task, update a record, draft a document) under the user's permissions.	A bypass of permissions or business rules. Every action runs as the signed-in user (delegated) and is subject to the same authorization as the app itself.
Workspaces	A tenant-bounded scratch space where a long-running or multi-step agent persists files, memory, progress, and intermediate reasoning as it works — without that state ever leaving the Microsoft 365 trust boundary.	Giving multi-step agents a safe place to keep working state across turns — the enabler for long-running agents.	An access-control boundary or a permission system. It does not decide what work context the agent can read; Microsoft 365 permissions and sensitivity labels do that, always trimmed to the signed-in user.

Protocols (how you reach Work IQ): A2A (preview) — discover the Work IQ agent from its **agent card** (`workiq agents get-card`) and send it natural-language questions over JSONRPC for grounded answers; **MCP (GA)** — call the generic entity/copilot/schema tools for structured fetch-and-act workflows; **REST (preview)** — direct API for custom orchestration. The most compelling demos light up **Context + Chat + Tools + Workspaces** over the **same** work-context architecture — one work graph, four ways to consume it, with the user's permissions honoured every time.

Quick Reference — Work IQ Access Patterns

Authorization (all paths). Work IQ is **delegated-only** — every call runs as the signed-in user. Acquire a token for the Work IQ resource with the `WorkIQAgent.Ask` scope; on-behalf-of (OBO) is supported for service-mediated agents. Admin consent for the scope and Work IQ enablement in the Microsoft Admin Center are tenant prerequisites.

A2A — discover the Work IQ agent, then ask it a grounded work question (preview CLI):

```
# A2A is exposed through the PREVIEW Work IQ CLI (the `agents` commands are not in the stable build):
npm install -g @microsoft/workiq@preview          # or: npx -y @microsoft/workiq@preview <cmd>

workiq agents list                                # discover available agents (e.g. bizchat-as-gpt-scenar
workiq agents get-card --agent-id <agent-id>      # fetch that agent's A2A agent card
```

```
// The agent card (A2A spec) describes the endpoint your agent calls. Real shape:
{
  "protocolVersion": "0.3.0",
  "preferredTransport": "JSONRPC",
  "url": "<agent endpoint from the card>",    // e.g. the local relay prints http://localhost:5100/
  "skills": [
    { "id": "ask_work_iq", "name": "Ask Work IQ",
      "inputModes": ["text/plain"], "outputModes": ["text/plain"] }
  ]
}
```

```
# Audience: api://workiq.svc.cloud.microsoft  Scope: WorkIQAgent.Ask  (DELEGATED user token)
# 1) GET the agent card via `workiq agents get-card` (above).
# 2) Acquire a delegated token for the WorkIQAgent.Ask scope.
# 3) POST an A2A `message/send` JSONRPC request to the card's `url` with the user's work question;
#    the response is a synthesised answer with citations back to the meetings, emails, and files.
# Most teams start with MCP (`ask`) — it ships in the stable CLI; reach for A2A when you need
# agent-to-agent delegation to the Work IQ agent.
```

MCP — call Work IQ's generic tools (fetch context, then act):

```
# Model Context Protocol: connect to the Work IQ MCP server and call its generic tools.
# Tool categories: Entity (fetch / create_entity / update_entity / delete_entity /
#                   do_action / call_function), Copilot (ask / agents list),
#                   Schema (get_schema / search_paths).
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import MCPTool
# mcp = MCPTool(server_label="work-iq", server_url="<work-iq-mcp-endpoint>",
#               allowed_tools=["ask", "fetch", "create_entity", "update_entity"])
# Build an agent with mcp.definitions / mcp.resources; the agent calls Work IQ as a tool,
# reads context via `ask`/`fetch`, and acts via `create_entity`/`update_entity`/`do_action`.
```

Always use delegated user identity and least privilege — request only `WorkIQAgent.Ask`. Never hardcode tokens. Never widen the user's access; Work IQ answers are always permission-trimmed to the signed-in user.

Alternate Path — The Local Work IQ Simulator (run unblocked, no tenant required)

Several challenges depend on **tenant-level authorization** (Copilot licensing, admin consent for `WorkIQAgent.Ask`, Work IQ enablement in the Microsoft Admin Center, Dataverse environments, and connectors). If your team cannot obtain these in time, **you are not blocked**. Build a **Local Work IQ Simulator** — a small local **MCP server** that *imitates the real Work IQ MCP surface* — and point your agent at it instead of the live endpoint. Because your agent talks to Work IQ through a stable tool contract (the MCP tool categories, or — if you simulate A2A — an **agent card** at `/.well-known/agent-card.json` advertising a JSONRPC `ask_work_iq` skill, matching the real card shape), swapping the live endpoint for the simulator is a configuration change, not a rewrite. When you later get tenant access, repoint the agent and the demo runs unchanged.

What the simulator must do (all challenges)

- **Expose the same tool surface** your agent expects: the **Copilot** category (`ask` , `agents list`), the **Entity** category (`fetch` , `create_entity` , `update_entity` , `delete_entity` , `do_action` , `call_function`), and the **Schema** category (`get_schema` , `search_paths`).
- **Serve from local synthetic work context** — a folder of fixture files representing the work signals (emails, meeting recaps/transcripts, Teams messages, files, a people directory) plus JSON tables standing in for the Dataverse/connector entities. (*Synthetic data generation is a separate task — this pack only specifies the data types each challenge needs.*)
- **Return grounded, cited answers** to `ask` : synthesise across the relevant fixtures and return citations that point back to the specific fixture items (meeting id, email id, file id), mirroring the real Work IQ citation shape.
- **Honour a simulated permission model**: accept a "current user" / persona parameter and trim results to what that persona may see — so the **permission-trimming governance demo works locally**.
- **Persist Tools writes** to the local JSON tables so `create_entity` / `update_entity` are observable in the demo (and idempotent on re-run).

The simulator imitates Work IQ's *behaviour and contract*, not its scale or intelligence. It is a demo harness, not a product. State clearly in your pitch which capabilities ran on the simulator versus the live tenant.

Per-challenge simulator behaviour (the data types each must serve)

§C1 — Northbridge Health Network (cross-committee coordination). Fixtures: **meeting** recaps for *two distinct committees* (quality, credentialing) with decisions + action items, vendor **emails**, per-clinic **Teams channel** posts, a shared **corrective-action tracker** file, a **people** directory (committee membership), and a JSON **CAPA tracker**. **ask** must stitch decisions across both committees; **update_entity** must change corrective-action status in the JSON and surface past-due flags. No PHI/clinical content — administrative only.

§C2 — Contoso Precision Parts (program handover). Fixtures: program **emails** (customer escalations, supplier threads), **design-review meeting** recaps + transcripts with action items, **Teams channel** posts (Inconel-718 supplier), a **people** directory (owners, expertise), program **files** (PPAP qualification plan, risk register), and a JSON **milestone tracker** (milestone, status, owner, risk). **ask** must synthesise a handover brief across all of these; **create_entity** must write new risk items into the milestone-tracker JSON and show them on re-query.

§C3 — Meridian Group (engagement 360). Fixtures: client **emails** (escalations, scope), standup **meetings** + per-client **Teams channels**, SOW/deliverable **files**, a **people** directory, a JSON **engagement tracker** (sprint, deliverable, due date, status) and a JSON **PSA** table (budget, utilisation). **ask** must produce an account 360 and a portfolio-wide risk scan; **create_entity** must add a change-request record. Persona parameter scopes which engagements a user can see.

§C4 — Arundel Holdings (facilities coordination). Fixtures: facilities coordination **meeting** recaps + transcripts (decisions, actions), engineering cause-analysis **emails**, building-ops **Teams channel** (contractor dispatch), a shared **works log** file, a **people** directory, and a JSON **work-order tracker** (work order, asset, building, status, owner, due date), with simulated ServiceNow/Maximo rows. **ask** must build the current operational picture *and* a portfolio vendor scan; **create_entity** / **update_entity** must create/update work orders and show the changes on re-query.

§C5 — Westbrook University (academic governance). Fixtures: **meeting** recaps for *two distinct committees* (curriculum, program-review) with decisions + action items, accreditation-liaison **emails**, per-program **Teams channel** posts, a shared **self-study** document file, a **people** directory (committee membership), and a JSON **accreditation/action tracker**. **ask** must stitch decisions across both committees into a site-visit readiness brief; **create_entity** must add outstanding tasks. No FERPA-protected student records — administrative governance only.

§C6 — Enterprise Developer Knowledge Hub (incident shift handover). Fixtures: incident-bridge **meeting** recaps + transcripts (decisions, actions), SRE root-cause **emails**, #atlas-incident **Teams channel** (mitigations), a shared **incident doc** file, a **people** directory, and a JSON **incident & action tracker** (action item, service, status, owner, due time), with simulated Jira/PagerDuty rows. **ask** must build the current operational picture *and* a chronological post-incident timeline; **create_entity** / **update_entity** must create/update action items and show the changes on re-query.

Testing your simulator (a three-layer checklist)

Don't wait until the demo to find out your simulator is wrong. Verify it in three escalating layers — each layer catches a different class of failure, and you should be green at every layer before you build the agent on top.

Layer 1 — Data integrity (does the corpus hold together?) Run a scenario validator that loads your fixtures and asserts the contract every scenario must satisfy, *before* any agent is involved:

- Every golden/expected question matches exactly one scripted answer (no two questions collide, none fall through to the fallback unintentionally).
- Every citation id referenced by an answer resolves to a real fixture item (no dangling citations).
- Every restricted answer ships a redacted variant, and at least one persona is actually blocked from at least one cited source (your RBAC demo is wired, not aspirational).
- `fetch` / `create_entity` / `update_entity` round-trip on each Tools table (create appends and is indexed; update patches and is observable on re-query; ids don't collide).

Layer 2 — Engine behaviour (does `ask` answer correctly per persona?) Drive the engine directly with a human-readable tester. For each challenge question, confirm: the response is grounded in the expected sources; the citation list contains the right fixture ids with `source_index`, `title`, and `kind`; and the **same question asked as two different personas produces a full answer for the privileged persona and a redacted answer + governance note for the under-privileged one**. This is the money shot of the governance demo — rehearse it.

Layer 3 — MCP wire contract (does a real client see the right tools?) Register the simulator as an MCP server in your agent client (or a generic MCP client) and confirm over the wire: the tool list advertises your expected surface; `ask_work_iq` accepts `{question, fileUrls?}` and returns the `{response, conversationId, citations}` shape; and a Tools call (`update_entity` / `create_entity`) succeeds and is reflected on a follow-up `fetch`. If you simulate A2A instead of MCP, fetch the `/.well-known/agent-card.json` and confirm the `ask_work_iq` skill is advertised with the right shape.

Per-question acceptance criteria (apply to every tier of your capability table):

Tier	A question passes when...
Work Context — single signal	The answer is grounded in exactly the one expected signal and cites it; no fabricated cross-references.
Work Context — multi-signal	The answer stitches ≥ 2 distinct signal <i>types</i> and the citation list contains at least one id from each.
Agentic synthesis	The answer spans ≥ 3 sources, and for an under-privileged persona the restricted source is withheld with a governance note while the non-restricted synthesis still returns.
Agent with MCP / A2A	The agent reads via Context <i>and</i> performs the expected Tools write (<code>create_entity</code> / <code>update_entity</code>), and the write is visible on a re-query.

Reference implementation. All six scenarios ship fully built with the simulator — **\$C1 Northbridge Health Network** (`update_entity` hero action), **\$C2 Contoso Precision Parts** (`create_entity` hero action), **\$C3 Meridian Group**, **\$C4 Arundel Holdings**, **\$C5 Westbrook University**, and **\$C6 Enterprise Developer Knowledge Hub**. Each ships a `golden.json` of 8 questions with persona-scoped answers, and all scenarios share a single data-driven engine — so building your own variant is **data only — no engine code**. The three layers above map to the bundled `validate_scenario.py` (Layer 1), `demo.py` (Layer 2), and the end-to-end servers `tests/mcp_e2e.py` (MCP) and `tests/a2a_e2e.py` (A2A, all six scenarios) (Layer 3). Run them for your scenario and get them green before you demo.

Good luck. Ground your agents in the work that's already happening — and answer what no single app can see.